

Module 13 — FastAPI Service (Backend)

app/app_fastapi.py • 465 lines

In one sentence

Serves recommendations over HTTP. Transport only - no scoring logic lives here.

1. What this module does

This turns the recommender into something other systems can actually call. A website, a mobile app or an email campaign sends an HTTP request and gets back ranked products with explanations.

The important architectural point: **this file contains no recommendation logic**. It loads the HybridRecommender service object and calls it. All it does is validate requests, shape responses and map errors. The dashboard loads exactly the same object, which is why the two can never give different answers for the same customer.

2. Concepts covered

Every idea this module uses, in plain terms.

Concept	In simple terms	Where it is used here
REST API	Send a URL, get data back	The whole module
FastAPI	A modern Python web framework	The framework used
Pydantic models	Define and enforce the data shape	All the response classes
Request validation	Reject bad input automatically	Query(10, ge=1, le=50)
Response validation	Catch bad output before it ships	response_model on each route
Path and query parameters	/recommend/42?top_n=5	How requests are structured
Lifespan / startup event	Do slow setup once, not per request	The lifespan handler
HTTP status codes	200 OK, 404 not found, 422 bad input, 503 not ready	Error mapping
Separation of concerns	Transport separate from logic	The core design rule
Batch endpoint	Score many customers in one call	POST /recommend/batch
Health check	Is the service actually ready?	GET /health
OpenAPI / Swagger	Auto-generated interactive docs	Available at /docs

3. How it works, step by step

- 1. At startup, load the recommender once.** Loading several hundred MB of matrices per request would put response times in the tens of seconds.
- 2. A request arrives**, for example GET /recommend/3181?top_n=10.
- 3. FastAPI validates it automatically** - asking for 999 items returns 422 with no hand-written check.
- 4. Call the recommender** and get back a DataFrame.
- 5. Build an explanation** for each item.

6. **Shape the response** through the Pydantic models, adding business context such as total basket value.

7. **Return JSON** including the strategy label.

4. Endpoints

Endpoint	What it returns
GET /	Service metadata and endpoint list
GET /health	Readiness, and whether the NCF actually loaded
GET /recommend/{user_id}	Top-N with explanations and signal breakdown
GET /similar-items/{item_id}	Content-similar products
GET /explain/{user_id}/{item_id}	The full three-part explanation
GET /users/{user_id}	Customer profile and cold-start status
GET /items/{item_id}	Product detail and long-tail flag
POST /recommend/batch	Many customers in one call, for offline jobs

5. Inputs, outputs and how to run

	Detail
Inputs	HTTP requests
Outputs	Validated JSON
Docs	http://localhost:8000/docs
Startup	A few seconds to load artifacts
Run with	<code>python app/app_fastapi.py</code>

6. Key code

app/app_fastapi.py — lifespan()

```

195 | async def lifespan(app: FastAPI):
196 |     """Load all model artifacts once at startup."""
197 |     global recommender
198 |
199 |     print("Loading recommendation artifacts ...")
200 |     try:
201 |         recommender = HybridRecommender.load()
202 |         print("  users      : {:,}".format(len(recommender.users_df)))
203 |         print("  items      : {:,}".format(len(recommender.items_df)))
204 |         print("  interactions : {:,}".format(len(recommender.interactions_df)))
205 |         print("  NCF loaded  : {}".format(recommender.ncf_model is not None))
206 |         print("Service ready.")
207 |     except FileNotFoundError as exc:
208 |         # Do not start pretending to be healthy. A recommender missing its
209 |         # artifacts should refuse requests, not serve degraded results silently.
210 |         print("FAILED to load artifacts: {}".format(exc))
211 |         print("Run the pipeline first - see README Quickstart.")
212 |         recommender = None
213 |
214 |     yield
215 |
216 |     print("Shutting down recommendation service.")

```

Loads everything once, and refuses to pretend to be healthy if it cannot.

7. Things to remember

A real bug this design prevents

The previous version loaded the NCF from a folder that never existed, inside a bare try/except. The service started, reported itself **healthy**, and served every single recommendation with the deep-learning signal switched off. Now a missing model prints the exact file path and the fix command, and /health reports ncf_available explicitly.

Why Pydantic validates the output too

If a future scoring change starts emitting nulls where a price is expected, the response fails at the API boundary rather than reaching the storefront and breaking a page.

8. Likely questions

Q. Why does the API contain no recommendation logic?

A. So the API and the dashboard cannot drift apart. Both load the same HybridRecommender object. The previous version duplicated the fusion logic into the API file, and kept a copy of every module in a second folder just to satisfy imports - which is exactly how two surfaces start giving different answers.

Q. Why load the model at startup rather than per request?

A. The artifacts are several hundred megabytes. Loading per request would make every call take tens of seconds. Loading once at startup means a request is about 190 ms.

Q. What does the strategy field do?

A. It tells the caller which path produced the answer - hybrid_fusion, cold_start_user_profile, or global_popularity_fallback. A client must be able to tell a personalised result from a fallback, otherwise a UI can end up claiming a brand-new customer has a learned taste profile.

Q. Why is there a batch endpoint?

A. For offline work - nightly emails, precomputed homepage slots. Doing those one HTTP request at a time would make the round trips dominate the runtime.